

Tài liệu Module Pipeline Theo dõi Sự Chú ý

Lê Hoàng An

May 31, 2025

1 Giới thiệu

Tài liệu này mô tả các luồng xử lý của module pipeline theo dõi sự chú ý, bao gồm ba hàm chính: `create_complete_pipeline`, `demo_real_time_monitoring`, và `main`. Module này tích hợp các thành phần (`FaceDetector`, `FaceFeatureExtractor`, `SyntheticDataGenerator`, `AttentionClassifier`, `AttentionMonitoringSystem`) để tạo một hệ thống hoàn chỉnh từ tải dữ liệu, huấn luyện mô hình, đến đánh giá và theo dõi thời gian thực. Tài liệu bao gồm các luồng chính, vòng đời (lifecycle), luồng xác thực đầu vào, và các luồng chi tiết liên quan đến pipeline.

2 Flow Chính của Module

Module pipeline hoạt động theo các bước chính sau:

1. **Tải dataset:** Tải ảnh từ đường dẫn dataset bằng `DatasetLoader`.
2. **Tạo dữ liệu tổng hợp:** Sử dụng `SyntheticDataGenerator` để tạo dữ liệu huấn luyện.
3. **Huấn luyện mô hình:** Huấn luyện và so sánh các mô hình (`random_forest`, `svm`, `neural_network`) bằng `AttentionClassifier`, chọn mô hình tốt nhất.
4. **Khởi tạo hệ thống:** Tích hợp các thành phần vào `AttentionMonitoringSystem`.
5. **Đánh giá hiệu suất:** Phân tích hiệu suất hệ thống trên dataset.
6. **Tạo báo cáo:** Tạo báo cáo tổng hợp và lưu kết quả.
7. **Demo thời gian thực:** Theo dõi trạng thái chú ý qua webcam (tùy chọn).

3 Lifecycle của Module

Vòng đời của pipeline bao gồm các giai đoạn sau:

1. **Khởi động (main):**

- Kiểm tra đường dẫn dataset.
- Gọi `create_complete_pipeline` để thực hiện toàn bộ pipeline.
- Hỏi người dùng về demo webcam và chạy `demo_real_time_monitoring` nếu cần.

2. **Tạo pipeline (`create_complete_pipeline`):**

- Tải dataset bằng `DatasetLoader`.
- Tạo dữ liệu tổng hợp bằng `SyntheticDataGenerator`.
- Huấn luyện các mô hình bằng `AttentionClassifier`.
- Lưu mô hình tốt nhất.
- Khởi tạo `AttentionMonitoringSystem`.
- Phân tích dataset và lưu báo cáo.

3. **Demo thời gian thực (`demo_real_time_monitoring`):**

- Mở webcam và xử lý các frame liên tiếp.
- Hiển thị kết quả và tạo báo cáo demo.

4. **Kết thúc:** Giải phóng tài nguyên (webcam, cửa sổ OpenCV) và lưu kết quả.

4 Input Validation Flow

Luồng xác thực đầu vào được thực hiện như sau:

1. **Hàm `main`:**

- Kiểm tra `DATASET_PATH`: Nếu không tồn tại, in thông báo lỗi và thoát.
- Kiểm tra `demo_choice`: Chuyển đổi thành chữ thường và kiểm tra giá trị y/n.

2. **Hàm `create_complete_pipeline`:**

- Kiểm tra `dataset_path`: Xử lý lỗi nếu không tải được dataset.
- Kiểm tra `model_save_path`: Tạo thư mục chứa nếu chưa tồn tại bằng `os.makedirs`.
- Kiểm tra dữ liệu tổng hợp: Giả định dữ liệu từ `SyntheticDataGenerator` hợp lệ.
- Kiểm tra huấn luyện mô hình: Bắt ngoại lệ cho từng mô hình và tiếp tục nếu có lỗi.
- Kiểm tra kết quả huấn luyện: Nếu không có mô hình nào thành công, trả về None.

3. Hàm **`demo_real_time_monitoring`**:

- Kiểm tra webcam: Nếu không mở được, in thông báo lỗi và thoát.
- Kiểm tra frame: Nếu không đọc được frame, in thông báo và thoát.
- Kiểm tra thời gian: Thoát khi đạt `duration_seconds` hoặc nhấn phím q.

5 Các Flow Chi Tiết

Dưới đây là các luồng chi tiết liên quan đến các hàm chính:

5.1 Tạo pipeline hoàn chỉnh (**`create_complete_pipeline`**)

- **Đầu vào:** `dataset_path`, `model_save_path`.
- **Xử lý:**
 - **Tải dataset:**
 - * Khởi tạo `DatasetLoader` và tải ảnh với giới hạn `max_images_per_person=max_people=100`.
 - * Xử lý lỗi nếu tải thất bại.
 - **Tạo dữ liệu tổng hợp:**
 - * Khởi tạo `SyntheticDataGenerator` với `seed=42`.
 - * Tạo 5000 mẫu với phân phối [45, 30, 25].

- * Trực quan hóa dữ liệu và phân tích tương quan.
- **Huấn luyện mô hình:**
 - * Thử các mô hình `random_forest`, `svm`, `neural_network`.
 - * Huấn luyện với `optimize_hyperparams=True` và `validation_split=0.2`.
 - * Lưu kết quả và chọn mô hình có độ chính xác cao nhất.
 - * Lưu mô hình tốt nhất vào `model_save_path`.
- **Khởi tạo hệ thống:**
 - * Khởi tạo `AttentionMonitoringSystem` với `mtcnn`, `mediapipe`, và mô hình đã lưu.
- **Phân tích dataset:**
 - * Gọi `analyze_dataset` với `sample_size=50`.
- **Tạo báo cáo:**
 - * Tạo `DataFrame` so sánh các mô hình.
 - * Tạo báo cáo JSON chứa thông tin dataset, dữ liệu tổng hợp, so sánh mô hình, và hiệu suất hệ thống.
 - * Lưu báo cáo vào thư mục `complete_analysis_results`.
- **Đầu ra:** Dictionary chứa `monitoring_system`, `best_model`, `dataset_loader`, `data_generator`, `analysis_results`, và `final_report`.

5.2 Demo theo dõi thời gian thực (demo_real_time_monitoring)

- **Đầu vào:** `monitoring_system`, `duration_seconds`.
- **Xử lý:**
 - Mở webcam bằng `cv2.VideoCapture`.
 - Lặp xử lý frame:
 - * Đọc frame và gọi `monitoring_system.process_frame`.
 - * Hiển thị frame đã xử lý bằng `cv2.imshow`.
 - * Thoát nếu đạt thời gian `duration_seconds` hoặc nhấn phím q.
 - Giải phóng webcam và đóng cửa sổ OpenCV.

- Tạo báo cáo demo bằng `monitoring_system.generate_report`.
- In thống kê: thời gian, số frame, FPS, tỷ lệ chú ý, điểm tập trung, và số lần chuyển trạng thái.
- **Đầu ra:** Dictionary báo cáo demo.

5.3 Hàm chính (main)

- **Đầu vào:** Không có.
- **Xử lý:**
 - Định nghĩa `DATASET_PATH` và `MODEL_PATH`.
 - Kiểm tra sự tồn tại của `DATASET_PATH`.
 - Gọi `create_complete_pipeline`.
 - Hỏi người dùng về demo webcam:
 - * Nếu chọn y, chạy `demo_real_time_monitoring`.
 - * Lưu kết quả demo vào thư mục `demo_results`.
 - In thông báo hoàn thành và hướng dẫn kiểm tra kết quả.
- **Đầu ra:** Không có (thực thi pipeline và in kết quả).

6 Kết luận

Module pipeline theo dõi sự chú ý cung cấp một giải pháp tích hợp để xây dựng, huấn luyện, và đánh giá hệ thống theo dõi trạng thái chú ý. Các luồng xử lý được thiết kế để đảm bảo tính mạnh mẽ, với xác thực đầu vào, xử lý lỗi, và trực quan hóa kết quả. Để sử dụng hiệu quả, cần đảm bảo các thư viện (`opencv-python`, `facenet-pytorch`, `mediapipe`, `scikit-learn`, `matplotlib`, `seaborn`, `pandas`, `numpy`) được cài đặt, đường dẫn dataset hợp lệ, và thiết bị webcam (nếu dùng demo) hoạt động.